

SafeEye: AI-based Safety Equipment Detection Using YOLOv8s

Team Name : Udbhav

Team Members : • Shweta Mishra (Team Lead)
• Samiksha Sharma

Institute : Accurate Institute of Management and Technology

Hackathon : CodeAlchemy Hackathon 2025

Track : Duality AI – Space Station Safety Object Detection

Abstract

This project implements a real-time safety equipment detection system using YOLOv8s. The model detects seven safety-related objects in space-station environments using the Duality Falcon synthetic dataset. The trained model achieved an mAP@50 of 0.55 and real-time inference (66 ms per image) on an RTX 3050 GPU.

Dataset

The dataset was given by Duality Falcon for the Space Station Safety challenge. It has images of different safety objects like fire extinguishers, oxygen tanks, first-aid boxes, and fire alarms in space-station environments.

There are 7 classes in total and all images have YOLO-format labels (text files with object coordinates).

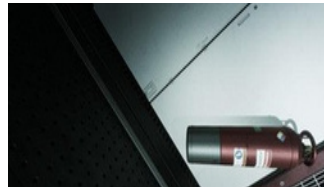
The data is divided into train, validation and test sets.

Sets	Purpose	% of Data
Train	Used for model learning	70%
Validation	Used to check accuracy during training	20%
Test	Used for final model testing	10%

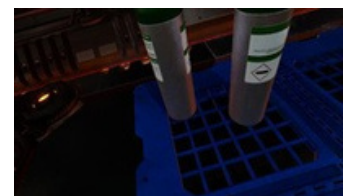
Below are a few sample images from the dataset.



Cluttered



Light



Dark

Dataset Setup:

- Source: Duality Falcon Synthetic Dataset
- Classes: 7 safety objects (OxygenTank, FireAlarm, etc.)
- Split Ratio: 70:20:10 (Train/Val/Test)
- Format: YOLO → [class, x_center, y_center, width, height]
- Variations: dark, light, cluttered scenes

Methodology

Model Architecture:

Framework	Ultralytics YOLOv8s
Hardware	NVIDIA RTX 3050
Key Features	CSPDarknet backbone, PANet neck, decoupled head

Training Configurations:

Parameter	Value
Image Size	640 x 640
Epochs	100
Optimizer	Adam
Batch Size	16
Learning Rate	0.001
Early Stopping	Enabled

→ The YOLOv8s model was trained for 100 epochs, monitored using validation metrics (mAP, precision, recall), and evaluated on unseen test images for performance consistency.

Work Flow

- Dataset divided into train, val, and test folders.
- YOLOv8s selected for its balance of accuracy and inference speed.
- Model trained on RTX 3050 GPU using Adam optimizer.
- Evaluation done using precision, recall, and mAP@50.

Challenges and Solutions

During model training and evaluation, a few technical and data-related challenges were encountered.

Each issue was analyzed and resolved through structured debugging and parameter optimization, as summarized below.

Challenge	Solution
Dataset path configuration errors	Fixed incorrect directory references in yolo_params.yaml updated script logic to correctly link train, val, and test folders.
Missed small or distant objects	Increased training epochs from 50 → 100 and switched to th Adam optimizer for smoother gradient updates and better feature extraction
Low-light image performance	Utilized Falcon's dark and bright dataset variants and fine-tuned the learning rate (0.001) to achieve stable training across lighting conditions.
Overfitting during long training	Enabled early stopping (patience = 15) and monitored validation loss to automatically stop training once improvement plateaued.

These improvements helped stabilize validation loss and increased the model's overall mAP@50 from 0.45 to 0.55.

Post-adjustment, the YOLOv8s model achieved consistent detection accuracy and maintained real-time inference speed on the RTX 3050 GPU.

Before optimization: mAP@50 = 0.45 | After optimization: mAP@50 = 0.55

Model Optimizations

Hyperparameter Refinement :

The first few training runs (baseline) were conducted with conservative hyperparameters: EPOCHS = 10, MOSAIC = 0.4, OPTIMIZER = AdamW, MOMENTUM = 0.9, LR0 = 0.0001, and LRF = 0.0001. This configuration allowed rapid testing but limited learning depth and overall convergence.

Through iterative experimentation, the setup was refined as follows:

Parameter	Initial setting	Optimized Setting	Effect
Epochs	10	100	Longer training improved feature learning and stabilized validation metrics.
Optimizer	AdamW	Adam	Adam produced smoother updates and reduced gradient noise.
Learning Rate (LR0)	0.0001	0.001	Higher rate helped faster convergence without overshooting.
Early Stopping (Patience)	Not used	15 epochs	Prevented overfitting and reduced training time.

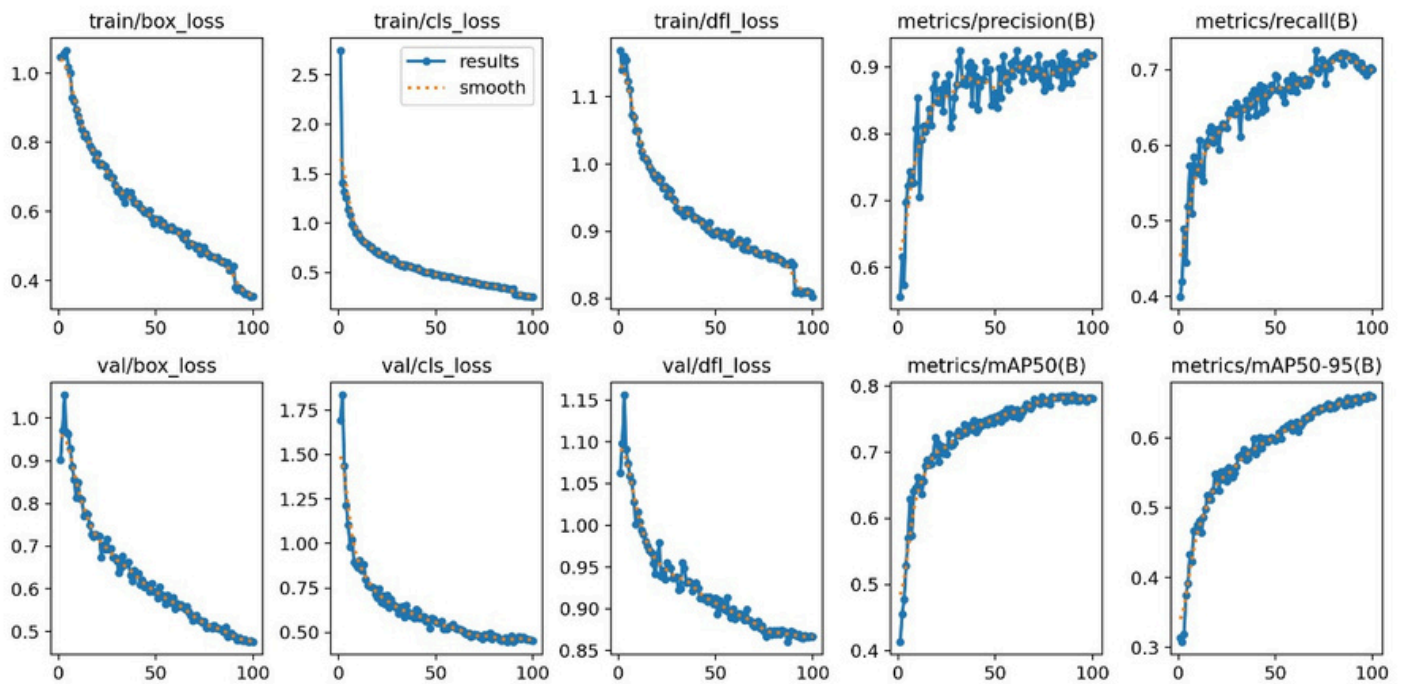


Figure: Improved mAP and loss trends after tuning.

Results After Optimization :

Metric	Before (10 epochs)	After (100 epochs)
mAP@50	0.42	0.55
Precision	0.60	0.66
Recall	0.47	0.55

The optimized setup achieved a +13-point gain in mAP and stabilized validation loss, with inference speed maintained at ~ 66 ms/image on the RTX 3050 GPU.

→ Through systematic tuning of learning rate, optimizer, and training duration, the YOLOv8s model achieved better convergence and higher detection accuracy while remaining real-time capable.

Performance Evaluation

The optimized YOLOv8s model was evaluated on the unseen test set to measure accuracy, robustness, and real-time efficiency. The evaluation focused on precision, recall, and mean average precision (mAP) scores.

Quantitative Metrics

Metric	Value	Description
Precision	0.665	Correct detec tions out of all predicted objects.
Recall	0.549	Fraction of actual objects successfully detected.
mAP@50	0.545	Main detection accuracy at IoU ~ 0.5
mAP@50-95	0.333	Mean precision across multiple IoU thresholds.
Interference Speed	~ 66 ms/image	Real-time processing speed on RTX 3050 GPU.

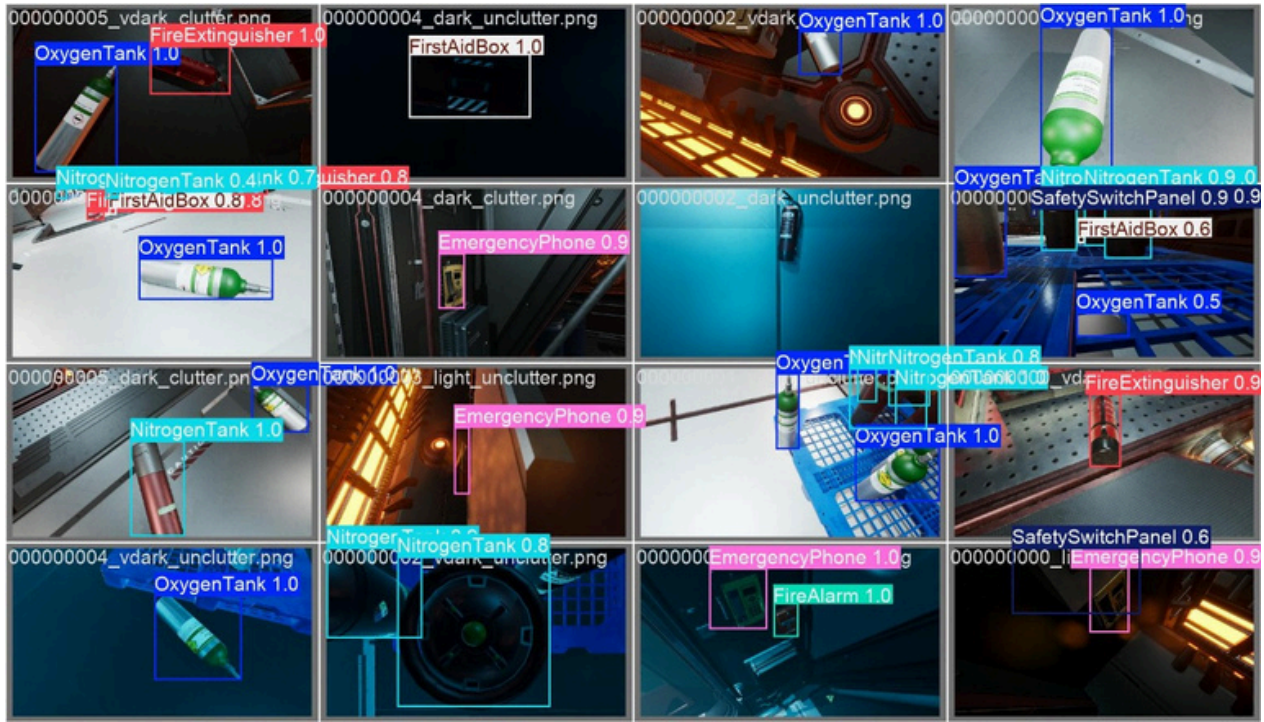
The results confirm strong generalization and real-time performance, achieving an overall mAP@50 ≈ 0.55 across all seven safety classes.

Failure Case Analysis

- Small FireAlarm instances in distant views were occasionally missed due to limited representation in the dataset.
- OxygenTank and NitrogenTank were occasionally confused because of similar appearance and color.
- Increasing dataset diversity or applying brightness-based augmentations can further reduce these errors.

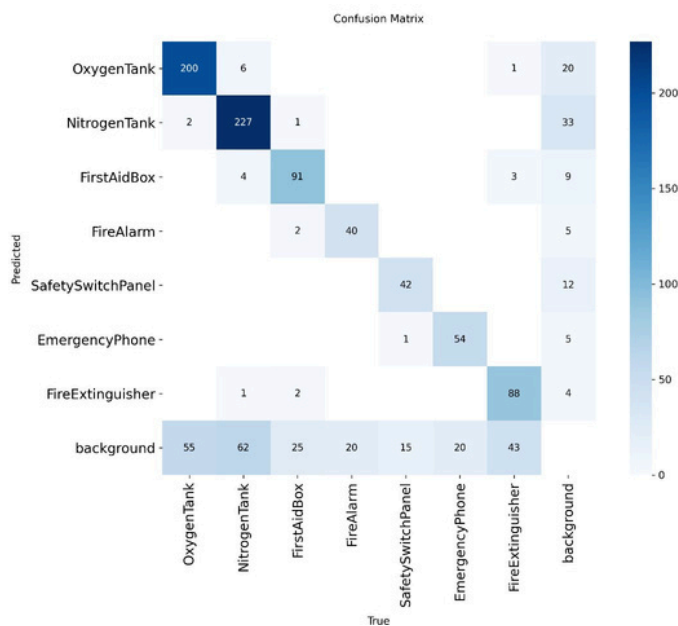
Visual Results

Sample Detections :



The model successfully detects all major safety equipment, even in low-light and cluttered conditions.

Confusion Matrix :



Displays class-wise prediction accuracy. Most misclassifications occurred between visually similar classes such as OxygenTank and NitrogenTank.

Conclusion & Future Work

The SafeEye project developed a YOLOv8s-based model to automatically detect seven types of safety equipment (fire extinguisher, alarm, oxygen tank, etc.) in space-station environments.

After optimization, the model achieved $mAP@50 = 0.55$, precision = 0.66, and recall = 0.55, with real-time speed (~66 ms/image) on an RTX 3050 GPU.

These results confirm reliable performance under both bright and low-light conditions.

Key Achievements :

- ✓ Stable training and improved accuracy after hyperparameter optimization.
- ✓ Real-time performance suitable for deployment on GPU and edge devices.
- ✓ Robust detection under lighting and background variations.
- ✓ Successfully automated identification of seven distinct safety classes.

Future Work :

- **Edge Deployment:** Convert to ONNX/TensorRT for Jetson or Raspberry Pi.
- **Live Monitoring:** Integrate with CCTV/webcam for continuous inspection.
- **Alert System:** Auto-notify when a safety item is missing.
- **Data Expansion:** Use Falcon's synthetic generator to retrain with new lighting and layouts.

App Integration :

The trained YOLOv8s model was deployed using a custom-built Streamlit application. The interface allows users to upload images or use a webcam to detect safety equipment in real time.

Detected objects are displayed with bounding boxes and class labels, while the sidebar provides model info and quick exit functionality.

The app runs locally and saves detection results for review, demonstrating real-world usability of the trained model.

